

ENVOPT: PERSISTENT ENVIRONMENT EVOLUTION FROM AGENT FEEDBACK

Anonymous authors

Paper under double-blind review

ABSTRACT

Data-centric self-improvement requires turning evidence about a model’s limitations into useful training experience. We study tool-using agents when target interaction traces are available but the target environment implementation is not. We present ENVOPT, which starts from an empty task-environment bank and maintains a persistent library of executable training programs. A fixed Planner combines source-linked trajectory analysis, measured task-linked requirement groups, versioned online rewards and maintained research experience to create, revise, retain or pause environments. Each program jointly defines task sampling, interaction and verification; bounded implementation checks precede fresh online RL without a separate student probe. Updated target behavior informs subsequent research. In preliminary single-seed Qwen3-4B studies, development-selected checkpoints score 42.08% on BFCL multi-turn test versus a matched 25.42% base, and 52.22% on native-tool-calling ACEBench-Agent versus 41.11%. The latter estimate has substantial small-test-set uncertainty. On τ^2 , the selected checkpoint reaches 39.33% test success but the last declines to 34.05%. These protocol-qualified trajectories demonstrate useful checkpoints alongside regressions; controlled comparisons are still needed to establish the contribution of persistent environment evolution.

1 INTRODUCTION

Data-centric recursive self-improvement (RSI) asks whether a researcher can turn evidence about a model’s limitations into more effective training experience. RSIBench-Data studies this question with a fixed researcher improving a separate learner through controlled training and evaluation services (Meng et al., 2026). Its discovery–reliability gap is consequential: a useful intervention may be found, yet subsequent revisions need not improve it. For tool-using agents, this research also requires an experience source that can execute actions, maintain state, and produce outcomes.

We study the setting in which the researcher observes permitted target tasks and interaction traces but cannot reuse the target backend or verifier implementation. A trace reveals only what happened along one interaction. To train on new actions and goals, the researcher must construct a runnable surrogate: which state persists, what each tool changes, what information each actor can obtain, and what constitutes the requested outcome. For example, a model may claim to update an artifact without executing the update, or apply a whole-record operation when only one field was authorized. Generating more conversations with similar vocabulary does not by itself recreate those decisions.

Executable environment synthesis and adaptive curricula provide important foundations (Song et al., 2026; Wang et al., 2026; Huang et al., 2026; Liu et al., 2026; Fang et al., 2026). The remaining problem is how to organize repeated research from target behavior. Three distinctions matter. First, a summary of a failure is an interpretation, not a measured capability score; the source interaction must remain inspectable. Second, an environment program is a reusable training asset, not a disposable collection of answers. Third, synthetic reward describes performance on the currently constructed tasks, while target evaluation measures whether training transfers. Conflating these signals can produce increasingly trainable but irrelevant experience.

We present ENVOPT, a feedback-guided method that maintains an evolving library of complete executable task environments. The studied runs start with an empty environment bank and generic interface examples. A persistent Planner uses source-linked trajectory analysis, fixed-development

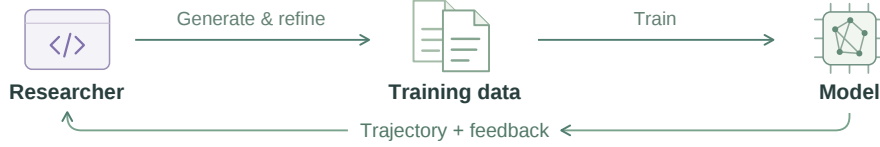
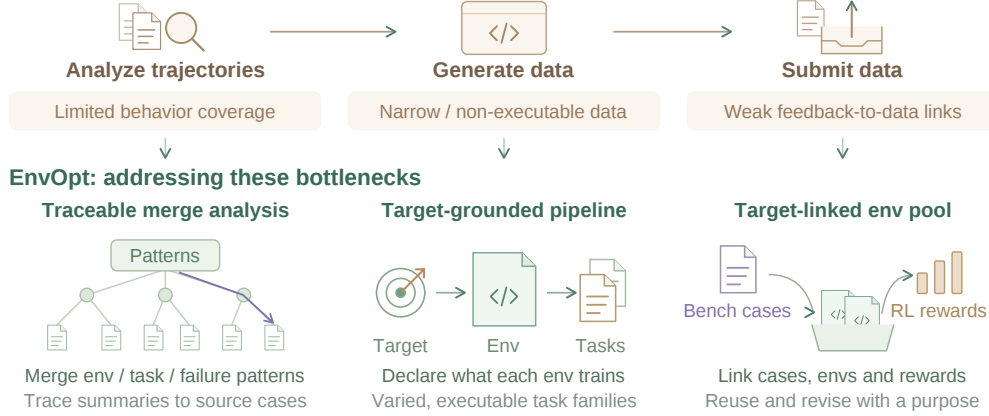
Data-centric RSI**Typical research workflow with native Codex**

Figure 1: Data-centric RSI and a typical native-Codex research workflow. ENVOPT addresses its bottlenecks with traceable analysis, target-grounded generation, and a target-linked environment pool.

outcomes, versioned online training feedback, and maintained research experience to choose which environments to create, revise, retain, or pause. Separate implementation workers translate those decisions into complete programs that jointly define task sampling, interaction, and verification. Fresh task instances support continuing online RL, whose outcomes inform the next research decision.

The design has two complementary adaptation axes: expand the library when a target mechanism is missing, and change the challenge or allocation of an existing mechanism when the learner’s needs change. A stable requirement map links research hypotheses to actual development-task IDs; scores for those groups are computed from recorded outcomes rather than written by a model. Candidate checks screen execution and discoverability without requiring a teacher solution or a pre-training student probe. The Planner can therefore retain a valid environment even when its current reward distribution is uninformative, revise it, or deliberately allocate no training to it.

Our contribution is this organization of environment research: evidence-linked diagnosis, persistent and selectively editable program assets, and a feedback loop that separates target outcomes from synthetic training signal. These components have precedents individually; their value must be tested through controlled comparisons, not inferred from interface names or successful execution. We report initial three-benchmark Qwen3-4B studies. BFCL shows substantial observed improvement, while τ^2 exhibits later regression and ACEBench shows gains alongside fluctuations on a small test set. These trajectories motivate evaluating the entire evolution process, not only its highest-scoring checkpoint.

2 RELATED WORK

Data-centric research for RSI. RSIBench-Data evaluates a frontier researcher that revises training-data strategies for a separate target model under shared service and resource controls (Meng et al., 2026). It supplies task-matched public seed repositories or examples; it does not prescribe a particular synthesis algorithm. Each candidate in its main setting uses LoRA SFT from a fixed base. ENVOPT studies a concrete executable-environment strategy, starting from an empty task-

environment bank and continuing the student’s online RL state. Both address the cross-model data-research component of RSI, not a claim that the student takes over or retrains the researcher. Autodata and DataMaster are further precedents for autonomous, downstream-feedback-driven data engineering (Kulikov et al., 2026; Du et al., 2026).

Executable environment synthesis. AgentScaler, EnvScaler, ScaleEnv, Agent World Model, and EnvFactory establish programmatic tools, synthetic state, task generation, and generated verification as training infrastructure (Fang et al., 2025; Song et al., 2026; Tu et al., 2026; Wang et al., 2026; Xu et al., 2026). FACET grounds task artifacts in a shared realized environment (Shi et al., 2026). We likewise bind requests and verification to concrete task data. Our emphasis is the persistent research process: complete environment programs retain identities and revisions, can be reused without code changes, and are reconsidered using target behavior and subsequent training feedback. Neither a database nor reusable task generation is claimed as a standalone invention.

Environment adaptation and self-play. EnvHarness reshapes an existing executable environment through Stage, Contract, and Chain components while retaining the original verifier; EnvRigger diagnoses policy behavior to author adaptations (Huang et al., 2026). Our researcher does not receive that target implementation and instead constructs independent training surrogates and their verifiers. This removes a reuse assumption but incurs additional modeling and verification risk. SPADE also generates complete stateful environments, including backend tool state, later user instructions, and state-based checks (Liu et al., 2026). Its designer and solver share a trained model, with corpus grounding, environment memory, and hint-based regret. ENVOPT uses a fixed external Planner, target-development traces, and explicit library-management decisions; the studied runs use no external grounding corpus. These are different information and optimization settings, not evidence of superiority over SPADE’s reported scores.

Curriculum and targeted synthesis. Unsupervised environment design and replay prioritize learning opportunities using regret or related signals (Dennis et al., 2020; Jiang et al., 2020). RODS uses online reward statistics to generate skill-aligned variants while preserving API dependency structure and managing a replay pool (Fang et al., 2026). Agent-World and AgentOmnia also adapt task coverage to diagnosed weaknesses (Dong et al., 2026; Jiang et al., 2026). ENVOPT additionally permits program-level changes to interaction and outcome semantics, while preserving interpretable target-task associations. Its reward-group statistics come from formal training, not an additional student admission test or a new gradient estimator. The Accuracy–Complexity–diversity framework and WRIT help explain why correctness, coverage, and relevant decision burden should be considered jointly (Zeng et al., 2026; Gu et al., 2026).

3 METHOD

ENVOPT alternates environment research and student learning (Figure 1). A fixed Planner maintains research context and issues bounded decisions; separate workers implement complete environments. Neither role may change the trainer, official evaluator, or sealed test data. The current representation replaces the earlier mandatory World–Scene hierarchy with a complete `Environment` interface.

3.1 RESEARCH STATE AND OBJECTIVE

Let π_{θ_t} be the student, o_t its optimizer state, $\mathcal{B}_t$ a versioned environment library, $\mathcal{C}_t$ an evidence-linked requirement map, and $\mathcal{M}_t$ maintained research experience. A plan P_t selects program revisions, sampler parameters η_j , and explicit task allocations n_j . Its realized training distribution is

$$q_t(x) = \sum_{j \in I_t} \frac{\tilde{n}_j}{\sum_{k \in I_t} \tilde{n}_k} q(x \mid E_j^{v_j}, \eta_j), \quad (1)$$

where I_t contains accepted, explicitly allocated environments and $\tilde{n}_j \leq n_j$ is the actual distinct-task count after construction. There is no implicit replacement allocation when an environment fails. The first library is empty; generic interface code and neutral examples are supplied, not a benchmark-specific task bank.

For fresh tasks D_t , the fixed training procedure $\mathcal{U}$ collects new online episodes and continues the student:

$$(\theta_{t+1}, o_{t+1}) = \mathcal{U}(\theta_t, o_t, D_t), \quad (2)$$

$$F_{t+1} = \text{Eval}_{\text{dev}}(\pi_{\theta_{t+1}}),$$

$$P_{t+1} = \text{Plan}(F_{t+1}, L_t, \mathcal{B}_{t+1}, \mathcal{C}_t, \mathcal{M}_{t+1}), \quad (3)$$

where L_t is version-linked formal-training feedback. These equations describe heuristic research decisions, not a differentiable meta-objective or a guarantee of monotonically improving scores. The trained student does not replace the Planner.

3.2 SOURCE-LINKED DIAGNOSIS AND MEASURED REQUIREMENT GROUPS

Permitted development trajectories are submitted for analysis with their public instructions, tool schemas, calls, observations, later requests, and official outcomes. Unavailable analyses are disclosed; original cases and scores remain accessible. Analysis distinguishes task demands, required environment mechanisms, and consequential agent behavior. Successful examples supply useful strategies; failures supply observed mistakes and qualified expected behavior. A failed task alone does not establish which capability is missing, and an agent-invented prerequisite must not become a task rule.

Reports are merged bottom-up. At every node, the model jointly reconciles domain names and summarizes their environment, task-behavior, and failure-behavior content. Source-item references are retained, and unique trajectory counts are computed by the controller rather than generated by the model. Compact summaries lead back to complete permitted cases and cross-checkpoint histories. Source coverage makes an interpretation inspectable; it does not prove that the summary preserved every important condition.

Separately, the Planner maintains stable requirement IDs with natural-language definitions and sets J_c of real development-task IDs. The controller computes their observed performance:

$$s_t(c) = \frac{1}{|J_c|} \sum_{i \in J_c} \frac{1}{K} \sum_{k=1}^K y_{t,i,k}, \quad (4)$$

where $y_{t,i,k}$ is the recorded task outcome. Definitions can be revised with provenance, in which case baseline and current outcomes are recomputed on the same membership. Groups may overlap; task success is not an isolated measurement of every requirement in a composite task. This map supplies a more stable comparison than counts under freely changing summary labels.

3.3 PERSISTENT PLANNING AND SELECTIVE LIBRARY EVOLUTION

The Planner inspects current versus previous and baseline development results, source cases, the environment catalog, and per-environment online reward distributions. Training observations are associated with the code revision and sampler parameters that produced them. They are not fresh measurements of a revised environment or the final policy.

For each chosen environment the action is *create*, *revise*, *retain*, or *pause*. Creation may cite a parent; revision preserves identity and publishes new code; retention uses the existing implementation with supported sampler parameters; pausing disables sampling while retaining code and history. Omitted environments receive no tasks that round. Horizontal expansion adds missing mechanisms; vertical adaptation changes dependencies, information burden, constraints, or instance workload. Either can simplify a task, but should not remove the decision that motivates it.

Every registered requirement receives an explicit train, monitor, or defer decision and a rationale. A train decision requires an associated positive allocation; this is an auditable planning contract, not a guarantee of semantic coverage or retained competence. The studied planning budgets allow at most ten create/revise decisions per round and one to ten instances per selected environment, with a total target of 96. Retain decisions do not consume this planned-edit budget; subsequent defect repair remains bounded.

The Planner maintains compact, revisable experience: what was attempted, what was observed, where a lesson applies, and what remains uncertain. Its model and session persist across research

turns, while its written experience is corrected rather than treated as an infallible log. The current decision-brief variant additionally records expected effects and explicitly reviews whether prior interventions landed and transferred. This stronger review contract is present in the native-ACE study but not the earlier two studies (Appendix B). An observed score increase is still an association, not a causal attribution to one program edit.

3.4 COMPLETE EXECUTABLE TASK ENVIRONMENTS

An environment is a complete module with a deterministic task sampler, episode construction, executable tools, and a reusable verifier. `make_task(seed, parameters)` fixes a request and JSON payload. `start(task)` creates independent mutable state and actor-visible interfaces from that payload. `check(task, state, trace)` evaluates the actual outcome. A Task binds the exact environment version, payload, sampling settings, and initial-information fingerprint. Reset does not call a model or resample a goal.

The same concrete goal data determines the request and checking target. Tools declare their schema and implementation together; the framework checks argument consistency, actor ownership and dynamic availability, records actual receipts, and restores state on declared rejection. User-owned actions execute on the same episode state, with private receipts disclosed only through legitimate interaction. Scripted later requests and model-backed users are both supported. State may be a database, virtual filesystem, or ordinary JSON structure; a separate business World is not required.

The studied cohort uses programmatic Boolean or component-based `Score` rewards. Component weights and mandatory guards are fixed before rollout; strict success requires all required outcomes and guards, whereas numeric reward can recognize partial progress. Checks should accept lawful alternative routes and distinguish a real effect from a completion claim. Shared data and typed execution narrow inconsistency risks but cannot prove arbitrary generated programs correct.

3.5 BOUNDED CONSTRUCTION, ONLINE LEARNING, AND FEEDBACK

Implementation workers receive the plan, relevant analysis, prior code when available, maintained experience, and interface examples. They emit a complete module, a change summary, and limitations. Candidate environments undergo isolated CPU checks of reconstruction, initial information, state isolation, tool effects, and verifier behavior. An independent model screens representative concrete instances for material executability or agent-side discoverability problems. Each candidate permits at most one code repair; accepted candidates are published independently. An unchanged, previously accepted code/parameter pair can reuse its review, but its new tasks still receive executable checks.

There is no teacher solution requirement, separate cold-start phase, pre-training student probe, or reward-variation admission gate. The final promoted library must reproduce the checked task semantics. Duplicate or failed candidates create explicit shortfalls, not replacement tasks from unplanned environments. A positive smaller batch is allowed, including a partial final optimizer batch; if no distinct accepted tasks remain, training is not submitted.

Formal RL generates eight episodes per Task in the reported studies. Rewards are normalized within the original task group even when episodes are segmented for training. Student-generated tokens, including retained reasoning tokens where applicable, determine policy loss; environment/user observations do not. Both weights and optimizer continue from the last completed training boundary. Fixed-development evaluation then supplies new evidence, while sealed test outcomes remain operator-only. Online all-one, all-zero, constant-partial, and mixed groups inform subsequent planning; none alone establishes task validity or useful transfer.

4 EXPERIMENTS

We report existing-run records, not new experiments executed during manuscript preparation. They include nine completed rounds each of Qwen3-4B-Instruct-2507 on τ^2 -Bench and BFCL multi-turn, and ten on native-tool-calling ACEBench-Agent. All three runs were stopped by the operator. ACE records were supplemented on 14 September 2026 at 17:43 UTC, completing test coverage through

Algorithm 1 ENVOPT: persistent research over executable environments

Require: Student (θ_0, o_0) , permitted dev evidence, generic interfaces/examples, budgets

- 1: Initialize empty library $\mathcal{B}_0$, requirement map $\mathcal{C}_0$, and experience $\mathcal{M}_0$
- 2: **for** $t = 0, \dots, T - 1$ **do**
- 3: Analyze current dev trajectories; merge with source links
- 4: Publish measured requirement groups and versioned training feedback
- 5: Persistent Planner inspects evidence; writes actions, allocations, and maintained experience
- 6: Implement selected create/revise actions; check all selected candidates with bounded repair
- 7: Publish accepted versions; construct exactly allocated distinct Tasks D_t
- 8: **if** $D_t = \emptyset$ **then**
- 9: Stop with retained failure evidence; do not submit training
- 10: **else**
- 11: Continue (θ_t, o_t) by online RL on D_t ; evaluate fixed dev
- 12: Preserve the completed program, task, learner, and feedback boundary
- 13: **end if**
- 14: **end for**

Table 1: Preliminary Qwen3-4B results. Select t^* using dev only (earliest maximum, with the base model eligible), then report its test score. “Last” means the final completed checkpoint: R9 for τ^2 /BFCL and R10 for ACE. Intervals resample paired test tasks within fixed subsets, retaining their three repeats; they do not measure training-seed variability.

Benchmark	t^*	Base (%)	Selected (%)	Δ pp [95% CI]	Last (%)
τ^2 -Bench	5	31.89 [†]	39.33	—	34.05
BFCL multi-turn	9	25.42	42.08	+16.67 [+12.3, +21.2]	42.08
ACEBench-Agent [‡]	3	41.11	52.22	+11.11 [−5.6, +28.9]	51.11

[†]Historical τ^2 test base uses the pre-repair replay protocol; no matched-protocol gain is claimed. [‡]ACE uses native tool calling and the two-category macro mean; all ten completed checkpoints have test measurements.

R10; the other two studies retain their earlier snapshot. Incomplete or cancelled training attempts are not plotted as additional rounds.

Evaluation and training. Fixed dev/test halves contain 139/139 τ^2 tasks (25 airline, 57 retail, 57 telecom per half), 400/400 BFCL tasks (100 per multi-turn subset), and 25/25 ACEBench-Agent tasks (10 multi-step, 15 multi-turn) (Barres et al., 2025; Patil et al., 2025; Chen et al., 2025). Each dev evaluation and each available test evaluation uses three repeats. We report task-weighted success for τ^2 and BFCL and the two-category macro mean for ACE; these are not pass@3 or full-leaderboard results. Test records are not fed to the Planner. All three studies begin with an empty environment bank, generic code examples, and no external grounding corpus.

Training uses full-parameter online GRPO, learning rate 3×10^{-6} , fixed-reference KL coefficient 0.005, eight episodes per Task, and at most eight Tasks per update, retaining tail batches. The per-round target is 96 Tasks, but exact allocations and construction failures can reduce the realized batch. Planner weights remain fixed (Astra); analysis, implementation, and screening use GPT-5.5; model-backed users use GPT-5.4. Each study has one evolution/training seed. Three evaluation repeats do not establish cross-seed method reliability. Appendix B records important protocol differences and retained anomalies.

Observed improvement and regression. We apply a dev-only selection rule to the saved trajectories: choose the earliest maximum over the base model and completed checkpoints. BFCL selects R9, scoring 42.08% on test versus 25.42% for its matched base model, an observed gain of 16.67 percentage points. Native-FC ACE still selects R3 after all ten rounds, scoring 52.22% versus 41.11%; its paired-task interval includes zero, reflecting the small 25-task test set. We do not select ACE’s higher R7 test score after seeing it. τ^2 selects R5 with 39.33% test success but falls to 34.05% at R9. Its historical 31.89% test base predates a replay repair, so we report no matched-protocol baseline gain.

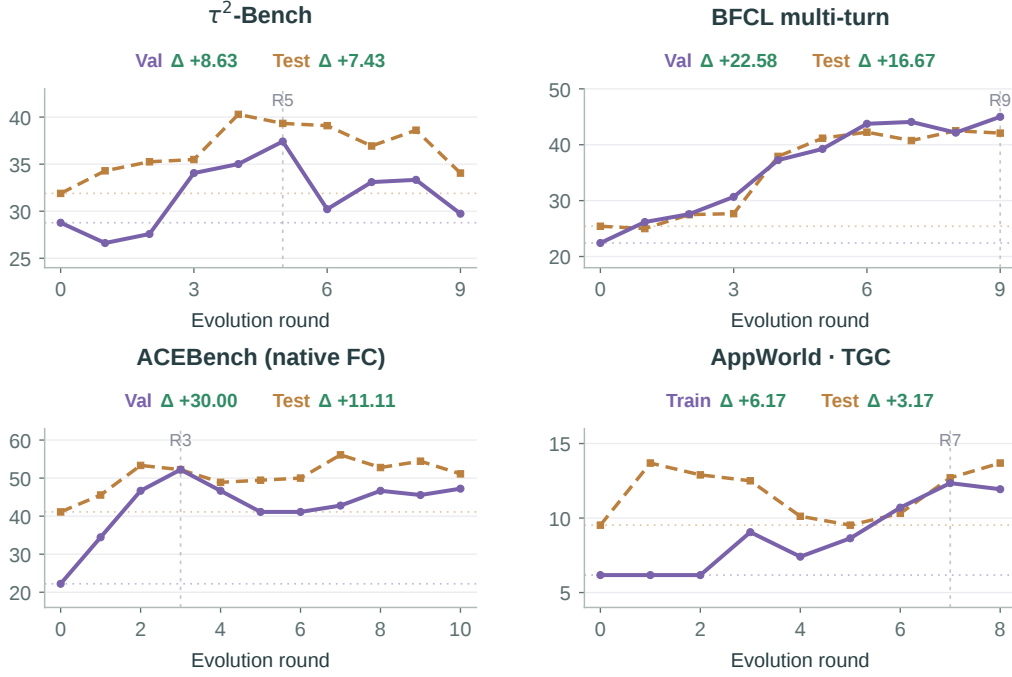


Figure 2: Evolution of τ^2 /BFCL/ACE (Val/Test, three repeats) and a separately supplied AppWorld run (Train/Test TGC, shown through R8). Solid/dashed lines denote the two splits; horizontal dotted lines mark their respective baselines at R0. Vertical guides select the earliest best Val checkpoint, or best train TGC for AppWorld; header Δ s use that checkpoint and each split’s own baseline (pp). Each panel uses its own round range and independently scaled percentage y-axis.

Figure 2 also shows why a single endpoint or synthetic reward curve is insufficient. BFCL’s improvement is not strictly monotone; τ^2 loses part of its intermediate performance; ACE’s test score reaches 56.11% at R7 and finishes at 51.11% in R10. The trajectories demonstrate that the implemented loop can produce useful checkpoints, not that environment revision caused the gains relative to equally budgeted fixed generators.

Controls still required. The decisive study should compare a frozen early library with fresh task resampling, sampler/allocation-only adaptation, and full program revision under matched training and research budgets. Further ablations should remove maintained experience, stale versus re-freshed target evidence, and the stable requirement map while preserving source access. A comparable SPADE-style or EnvHarness-style adaptation must disclose differences in target-code and development-feedback access. None of these controls has been measured in the present snapshot; we do not fill their cells with literature scores from different protocols.

Uncertainty and cost. Table 1 uses 10,000 paired bootstrap resamples of test tasks within fixed subsets, keeping each task’s three repeats together and preserving the benchmark aggregation. These intervals measure finite-test-set uncertainty conditional on a trained run, not training-seed variability. The recorded rounds contain 846, 809, and 626 distinct training Tasks for τ^2 , BFCL, and ACE respectively, with eight episodes per Task. Round counts do not fix training budgets: native ACE uses 96 Tasks in R1, 38 in R9, and 52 in R10. Final cost comparisons must also count analysis, planning, failed implementations, screening, serving, and evaluation, including queues and restarts. We make no end-to-end efficiency claim from task counts alone.

5 DISCUSSION AND LIMITATIONS

ENVOPT treats environments as persistent research assets and target trajectories as evidence for constructing them. This is a stronger modeling obligation than adapting a trusted target backend: observed traces underdetermine hidden state and unobserved actions. A program can execute correctly while training a simplified or irrelevant decision. Empty-bank initialization removes a supplied business environment, not the prior knowledge in language models, interface code, or generic examples.

Source-linked analysis is an investigation aid rather than an objective capability detector. Existing audits identify unsupported prerequisites inferred from the agent’s own errors, successful traces described too favorably, and changing domain granularity across rounds. Stable task membership prevents some counting ambiguities, but composite-task scores do not isolate individual skills. Likewise, aligned goal data and bounded solvability review do not guarantee a correct verifier; author and reviewer errors can be correlated.

The present results are preliminary, single-seed and protocol-qualified. They lack fixed-library and budget-matched adaptive controls; historical training anomalies and maintenance interventions are disclosed rather than erased. Development-conditioned environment research can overfit its feedback. Although test data are excluded from the Planner, repeated operator inspection of checkpoint tests limits claims of a pristine final holdout. A final confirmatory study should freeze the method and selection rule before using a fresh test set. Most importantly, later regressions show that discovering a useful checkpoint and sustaining reliable self-improvement remain different research outcomes.

AI USE STATEMENT

Generative AI assisted source-code and literature analysis, methodological formulation, drafting, and figure preparation. The research system uses language models for trajectory analysis, planning, program synthesis, solvability review and user simulation; generated-task rewards in the studied cohort are programmatic. Existing experiment records were read to prepare this draft; no new training or benchmark evaluation was run during manuscript preparation. **[TODO: The authors must verify this statement against the complete research history and review all AI-assisted claims and artifacts before submission.]**

REPRODUCIBILITY STATEMENT

The method interfaces, admission rule, and separation of development feedback from final evaluation are specified in Section 3. Appendix A records the experimental information needed for reproducibility. **[TODO: Add the frozen implementation revision, resolved run configurations, complete artifacts, and an anonymous code supplement after the experiments are finalized.]**

REFERENCES

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment. *arXiv preprint arXiv:2506.07982*, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Yuefeng Huang, et al. ACEBench: Who wins the match point in tool learning? *arXiv preprint arXiv:2501.12851*, 2025. URL <https://arxiv.org/abs/2501.12851>.
- Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design. *arXiv preprint arXiv:2012.02096*, 2020. URL <https://arxiv.org/abs/2012.02096>.

- Guanting Dong, Junting Lu, Junjie Huang, Wanjuan Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-World: Scaling Real-World Environment Synthesis for Evolving General Agent Intelligence. *arXiv preprint arXiv:2604.18292*, 2026. URL <https://arxiv.org/abs/2604.18292>.
- Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, Xinyu Zhu, WenHao Wang, Linfeng Zhang, Chen Qian, and Siheng Chen. DataMaster: Data-Centric Autonomous AI Research. *arXiv preprint arXiv:2605.10906*, 2026. URL <https://arxiv.org/abs/2605.10906>.
- Ruishan Fang, Siyuan Lu, Chenyi Zhuang, and Tao Lin. RODS: Reward-Driven Online Data Synthesis for Multi-Turn Tool-Use Agents. *arXiv preprint arXiv:2606.19047*, 2026. URL <https://arxiv.org/abs/2606.19047>.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, Shibin Wu, Zhengwei Tao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Towards General Agentic Intelligence via Environment Scaling. *arXiv preprint arXiv:2509.13311*, 2025. URL <https://arxiv.org/abs/2509.13311>.
- Hengrui Gu, Xiaotian Han, and Kaixiong Zhou. WRIT: Write-Read Intensive Trajectory Synthesis for Multi-Turn User-Facing Agents. *arXiv preprint arXiv:2606.02908*, 2026. URL <https://arxiv.org/abs/2606.02908>.
- Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiaxin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. EnvHarness: Awakening Static Worlds for Agent Learning. *arXiv preprint arXiv:2608.19880*, 2026. URL <https://arxiv.org/abs/2608.19880>.
- Hao Jiang, Gangtao Xin, Yingdi Huang, Guojie Zhu, Jiangshan Zhang, Xinyuan Lin, Yunkun Xu, Chengyu Shen, Wenlong Fei, Jiawei Li, Yujie Fu, Sichen Kang, Tingyu Xie, Yedi Hu, Jingren Zhang, Hongcheng Gao, Jianshu Zeng, Chong Chen, Chang Guo, Chao Feng, Feng Wang, Fulin Lin, Jinchao Ma, Lang Mei, Li Huang, Liyan Liu, Qing He, Shuting Tao, Siyu Mo, Xiangnan Chen, Xiaohan Yu, Xiaoyang Li, Yanheng Hou, Yanyu Wu, Zhihan Yang, Wentao Zhang, Yang Gao, and Zhao Cao. AgentOmnia: Scaling Agentic Models for Full-Scenario Applications. *arXiv preprint arXiv:2607.23124*, 2026. URL <https://arxiv.org/abs/2607.23124>.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized Level Replay. *arXiv preprint arXiv:2010.03934*, 2020. URL <https://arxiv.org/abs/2010.03934>.
- Ilia Kulikov, Chenxi Whitehouse, Tianhao Wu, Yixin Nie, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, Jakob Foerster, Xian Li, Han Fang, Sainbayar Sukhbaatar, and Jason Weston. Autodata: An agentic data scientist to create high quality synthetic data. *arXiv preprint arXiv:2606.25996*, 2026. URL <https://arxiv.org/abs/2606.25996>.
- Bo Liu, Simon Yu, Yiding Jiang, Ao Qu, Andrew Zhao, Zichen Liu, Junsu Kim, Zijian Zhou, Seungone Kim, Tongzheng Ren, Mickel Liu, Hanfei Yu, Zhaorun Chen, Weiyan Shi, Paul Pu Liang, Luke Zettlemoyer, Yejin Choi, and Natasha Jaques. SPADE: Self-Play in Adaptive Synthetic Executable Environments. *arXiv preprint arXiv:2608.19197*, 2026. URL <https://arxiv.org/abs/2608.19197>.
- Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. RSIBench-Data: Benchmarking data-centric research for recursive self-improvement. *arXiv preprint arXiv:2607.25886*, 2026. URL <https://arxiv.org/abs/2607.25886>.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.

- Kou Shi, Zun Wang, Qisheng Su, Shiting Huang, Ziao Zhang, Zhen Fang, Qingnan Ren, Jin Liu, Yu Zeng, Yiming Zhao, Lin Chen, Zehui Chen, and Feng Zhao. FACET: Preserving Source Intent and Executable State in Terminal Task Synthesis. *arXiv preprint arXiv:2608.18580*, 2026. URL <https://arxiv.org/abs/2608.18580>.
- Xiaoshuai Song, Haoifei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. EnvScaler: Scaling Tool-Interactive Environments for LLM Agent via Programmatic Synthesis. *arXiv preprint arXiv:2601.05808*, 2026. URL <https://arxiv.org/abs/2601.05808>.
- Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yi-Kai Zhang, Zhikang Xia, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, Qi Gu, Hui Su, and Xunliang Cai. ScaleEnv: Scaling Environment Synthesis from Scratch for Generalist Interactive Tool-Use Agent Training. *arXiv preprint arXiv:2602.06820*, 2026. URL <https://arxiv.org/abs/2602.06820>.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent World Model: Infinity Synthetic Environments for Agentic Reinforcement Learning. *arXiv preprint arXiv:2602.10090*, 2026. URL <https://arxiv.org/abs/2602.10090>.
- Minrui Xu, Zilin Wang, Mengyi DENG, Zhiwei Li, Zhicheng Yang, Xiao Zhu, Yinhong Liu, Boyu Zhu, Baiyu Huang, Chao Chen, Heyuan Deng, Fei Mi, Lifeng Shang, Xingshan Zeng, and Zhi-jiang Guo. EnvFactory: Scaling Tool-Use Agents via Executable Environments Synthesis and Robust RL. *arXiv preprint arXiv:2605.18703*, 2026. URL <https://arxiv.org/abs/2605.18703>.
- Xingshan Zeng, Zishan Xu, Boju Zhang, Yuzhou Wu, Lingzhi Wang, Jianghao Lin, Liangyou Li, Yasheng Wang, Lifeng Shang, Xin Jiang, Weinan Zhang, Yong Yu, Qun Liu, and Weiwen Liu. What Makes Good Agentic Data? An ACE Lens on Data Generation for LLM Agents. *arXiv preprint arXiv:2608.27260*, 2026. URL <https://arxiv.org/abs/2608.27260>.

A INTERFACES AND PROGRAM SEMANTICS

The current interface is a complete environment program, not a mandatory World–Scene stack. A module-level `build()` returns an `Environment`. Its sampler generates all initial data and goals before rollout; an `Episode` owns each rollout’s independent mutable state. Generated modules are not imported on the host as part of paper preparation.

Table 2: Responsibilities of a reusable environment program.

Interface	Responsibility
<code>make_task(seed, parameters)</code>	Deterministically return a request and JSON payload.
<code>start(task)</code>	Create fresh episode state and actor-visible instructions.
<code>@tool(..., actor=...)</code>	Declare matching schemas and executable methods.
<code>available(name, actor)</code>	Define dynamic actor-specific visibility.
<code>next_user / user_context</code>	Supply scripted follow-ups or legitimate model-backed user context.
<code>check(task, state, trace)</code>	Evaluate requested outcomes from actual state and execution.
<code>validate(reference)</code>	Assert selected tool/reward behavior, including meaningful rejection cases.

Task serialization binds the program-bank digest, request, payload, sampling parameters, and an initial signature covering state, public instructions, visible tools and user context. Repeated sampling with identical inputs must agree. Episode state is copied; expected tool rejection restores the previous state and records the result. These checks are consistency safeguards, not a proof that arbitrary Python or all unseen task instances are correct.

Reward. A generated task returns a Boolean or programmatic `Score`. With outcome components $c_k \in [0, 1]$, mandatory guards $g_\ell \in \{0, 1\}$ and positive pre-fixed weights w_k ,

$$R_x(\tau) = \left(\prod_\ell g_\ell \right) \frac{\sum_k w_k c_k}{\sum_k w_k}, \quad \text{Success}_x(\tau) = \mathbf{1}[\forall k : c_k = 1 \wedge \forall \ell : g_\ell = 1]. \quad (5)$$

The broader framework contains optional semantic-rubric support, but the studied task-generation contract excludes it. Model-backed users and independent solvability reviewers remain distinct from programmatic training rewards. Official target graders retain their own scoring logic.

Bounded checks. Changed programs undergo CPU execution checks and a model review of up to three representative constructed instances (first, middle, last), not an exhaustive semantic audit. At most one code repair is permitted. Retention can reuse the accepted review when code and sampler parameters match; all new tasks still receive executable checks. Promotion checks that request, payload, initial signature and sampling semantics remain unchanged. Failed candidates and exact duplicates produce explicit shortfalls, with no unplanned backfill.

B STUDY VERSIONS AND EVIDENCE BOUNDARIES

The earlier manuscript described World–Scene interfaces, curated World initialization and a separate student probe. Those describe a superseded protocol, not the results reported here. The three selected runs use `persistent_environment_v5`; their initial Git trees contain only an empty-bank seed manifest under `env/`. Generic interface examples remain provided.

The shared base model is Qwen3-4B-Instruct-2507, with RL temperature 0.7, 65,536-token context, 16,384-token response cap, task-group size eight, learning rate 3×10^{-6} , and fixed-reference KL coefficient 0.005. The recorded optimizer seed is 1234. Up to eight Tasks form one update; smaller tails are kept. The controller task configuration limits interactions separately from benchmark evaluation. Program construction is CPU work; pod counts refer to leased GPU jobs, with four GPUs per pod.

Recorded complete rounds report weight changes, finite training metrics, complete policy-token capture and task-group reward normalization. This does not make their training data defect-free:

Table 3: Frozen study identities and important differences. These are separate runs, not matched ablation arms.

	τ^2	BFCL	ACEBench-Agent
Run suffix	83b860	defa84	22571a
Launch (Beijing time)	Sep. 13, 02:25	Sep. 13, 02:25	Sep. 14, 09:46
Snapshot rounds	9, stopped	9, stopped	10, stopped
Test checkpoints	R1–R9	R1–R9	R1–R10
Task bank at start	Empty	Empty	Empty
Expectation/review contract	Earlier v5	Earlier v5	Decision brief v1
RL feedback access	Earlier diagnostic views	Earlier diagnostic views	Full group queries
Pods per training job	2	2	4
Recorded episode aborts	2/6,768	33/6,472	0/5,008

aborted episodes remain part of the audit, and maintenance/restart history is retained. No cancelled attempt or archived alternative checkpoint is substituted into a completed-round curve. The final confirmatory study should rerun a frozen, matched protocol with independent seeds.

Direct inspection of the retained rollout archives attributes 32 BFCL R2 aborts to an invalid model-backed user-context contract and one BFCL R1 abort to an empty user turn. The two τ^2 aborts are a user-tool interaction-budget exhaustion in R3 and an empty user turn in R8. These are not ordinary failed task outcomes or evidence that a task was simply too difficult. They qualify the historical training studies even though their subsequent fixed-target evaluations completed.

B.1 BENCHMARK PROTOCOLS

τ^2 -Bench. Each half has 25 airline, 57 retail and 57 telecom tasks. The revised fixed-seed partition stratifies airline/retail business-action labels and telecom issue/persona labels. Its baseline was re-partitioned from existing real evaluations without resampling responses; some earlier test-side records required reconstruction of static tool/schema wrappers, which is disclosed in their provenance. Overall is task-weighted, not an unweighted mean of domains. The evaluation policy is greedy and the user is GPT-5.4 at temperature zero.

Checkpoint test evaluation uses a replay repair that preserves the live behavior of unknown/rejected tools instead of allowing an exception to abort final-state reconstruction. Accepted test records use the corrected judge configuration and replay-v2 lineage; failed and superseded v1 evaluations are excluded. The historical test baseline predates the replay repair and is explicitly marked non-comparable in the dashboard registry. Figure 2 connects it for visual continuity and shows the stored-score difference; this is not a matched-protocol gain. Table 1 omits its gain estimate and paired interval.

BFCL. The fixed dev/test halves contain 100 tasks each from Base, Long Context, Missing Function and Missing Parameter. Legacy framework identifiers such as `bfcl_v3` are adapter names, not evidence that a full leaderboard suite was evaluated. The matched test base comes from the recorded September 6 collection with the same task membership, model, repeats and evaluation/serving configuration. The paper reports only these four multi-turn subsets.

ACEBench-Agent. The English Agent release is pinned to official source revision `56dd66c`. Its 20 multi-step and 30 multi-turn tasks are split by category and tool-class strata into equal dev/test halves (10+15 each), with seed 1234. We report the unweighted mean of the two category scores, matching the metric definition used in SPADE, not its complete evaluation protocol.

The studied run uses `acebench_native_fc_v1`: native structured tool calls and full append-only assistant/tool history, with the native business tools, task initialization, GPT-5.4 user, 40-event budget and official end-state grading. The audited schema projection corrects the documented required-field typo for `reserve_flight`; malformed arguments are not repaired to make episodes succeed. The 4B policy uses temperature 0.001 and a 1,000-token output budget. Multi-step outer prompts retain the wrapper’s Chinese compatibility setting while tasks/tools remain En-

glish; multi-turn prompts are English. These transport, history, user-model and serving details preclude presenting the scores as an exact SPADE reproduction.

The older September 13 ACE textual-interface study is not spliced into the native-FC curve. Its archived alternate training attempt is also excluded. Native test measurements cover all ten completed checkpoints. The later sweep reuses R1–R4 results and adds R5–R10 on the same 25 test tasks with three repeats and the same native-FC protocol. R6/R8 initially failed during remote toolchain detection before inference; the completed recovery results are included and the failed-start records are retained in provenance. Training was stopped during R11, which has no registered complete checkpoint and is excluded.

B.2 SNAPSHOT AND STATISTICAL CONSTRUCTION

The original snapshot at 14 September 2026, 16:04:42 UTC remains unchanged. A supplementary snapshot at 17:43:25 UTC (15 September 01:43:25 Beijing time) adds ACE R5–R10 test results and R10 development/training records. It retains the other two studies exactly, yielding nine completed rounds each for τ^2 /BFCL and ten for ACE. All included checkpoints have test results. Rebuilding the paper does not query or update source experiments.

Each plotted evaluation was independently recounted from stored per-task three-repeat outcome groups and checked against saved summary metrics. The updated snapshot records 218 source-file hashes, the prior snapshot’s hash, run/job/revision identities, actual task allocations, training audit fields, and links to baseline and operator-only test collections. The update checks that earlier ACE measurements and training records are unchanged; new test records match registered checkpoints and retain the fixed task membership. It contains outcome summaries, not a new evaluation. The paper scripts neither import generated environment modules nor invoke training, serving, grading or controller actions.

Selection uses the earliest maximum dev mean over R0 and the included completed checkpoints, with numerical ties rounded to twelve decimal places. It is an explicitly applied analysis rule, not a claim of preregistration. For paired test differences, each bootstrap draw samples tasks with replacement within their fixed subset, retains all three repeats for each sampled task, and applies the original aggregation: task-weighted for BFCL and a two-category macro mean for ACE. We use 10,000 draws with bootstrap seed 20260914. The τ^2 baseline mismatch excludes its paired baseline comparison.

C WITHIN-BENCHMARK EVOLUTION

D CONFIRMATORY COMPARISONS STILL NEEDED

Table 4: Unmeasured controls; no synthetic scores are supplied.

Control	Question isolated
Fixed library, fresh Tasks	Does later research improve over resampling and training alone?
Sampler/allocation only	Does executable program revision add value?
No maintained experience	Does accumulated, corrected research experience help?
Stale target evidence	Does fresh behavioral feedback change useful decisions?
No stable requirement map	Does explicit task-linked coverage improve retention or targeting?
Whole-program generation without persistence	Does retaining and revising assets improve cost or transfer?

Controls should share model initialization, target-information access and evaluation protocols. Compare both matched training episodes and matched total research budgets; count failed implementations, reviewer requests and all serving/evaluation overhead. Additional evidence should include independently audited accepted and rejected tasks, one traced intervention from source case to code change and subsequent outcomes, and repeated evolution seeds. Independent test tasks reserved after method freeze are needed for a final confirmatory result.

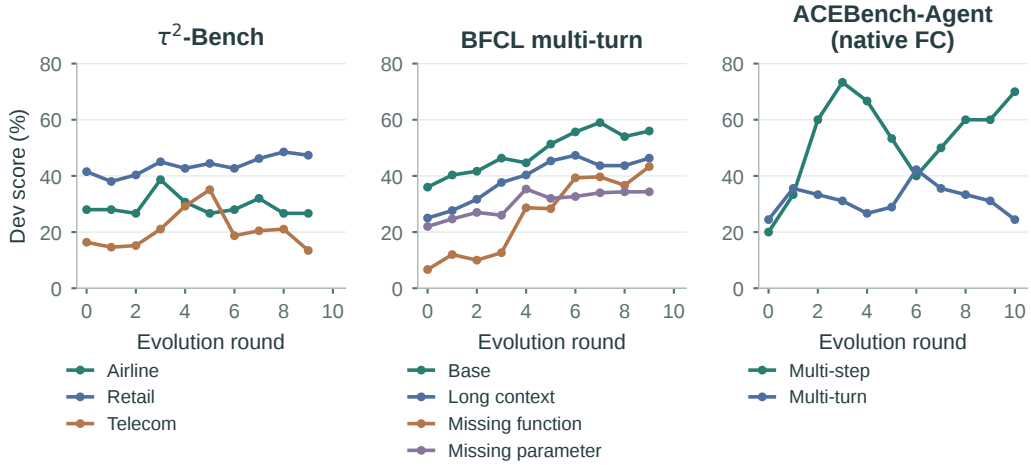


Figure 3: Fixed development-subset trajectories. These use the benchmark’s fixed labels, not the Planner’s changing analysis-domain names. BFCL’s Missing Function subset improves substantially; the later τ^2 decline is particularly visible in Telecom. Native ACE’s early gain is concentrated in Multi-Step. The figure describes associations within the recorded runs, not attribution to a particular environment revision.